MIS 3371 · SECTION 20300 · INSTRUCTOR WALKTHROUGH

PayDesk, Step by Step

Sixteen steps for the live demo - what to say, what to type, and what should happen

How to run this

Build the app in front of them in this order. The order is the teaching: CSS and JavaScript alternate, so something visible changes every few minutes and neither layer gets a boring half-hour.

Every code block below is lifted straight out of the finished app, so what you type is known to run. Type it; do not paste it. They are copying you, and paste teaches nothing.

Full run is 81 minutes. The cut-down path is on the next page if your session is shorter.

The running order

#	Layer	Step	TODO	Minutes
0	BOT	Open the finished app and the three empty files	-	3
1	CSS	Tokens first, then the page basics	C1 + C2	4
2	CSS	Layout: a container, a grid, and one card rule	C3 + C4 + C5	4
3	CSS	Make the form controls look like this decade	C6	4
4	JAV	The first function, and the trap inside it	J1	5
5	JAV	Validation: is this data usable at all?	J2	7
6	JAV	Read the form, and show the errors	J7 + J8	6
7	CSS	Now tell CSS what 'wrong' looks like	C7	3
8	JAV	Two questions the form cannot answer	J3 + J4	6
9	JAV	The business rules - and the order argument	J5	8
10	JAV	One payment, end to end	J6	6
11	JAV	Put the answer on the page	J9 + J10 + J11	7
12	CSS	One pill, three meanings	C8	3
13	JAV	Wire it up - and watch the whole thing come alive	J12	7
14	CSS	Make it survive a phone	C9	3
15	BOT	The two stretch pieces worth demoing	S1 + S3	5

If you only have 50 minutes

Keep: 0, 1, 2, 4, 5, 6, 7, 9, 10, 13. That is the full loop - tokens, layout, the Number('') trap, validation, the error round trip, the rules argument, and the submit handler that makes it live.

Cut: 3, 8, 11, 12, 14, 15. Tell them you are cutting them and that the code is in the reference folder. Naming the cut is the difference between pacing and running out of time.

Never cut step 9. The argument about branch order is the one thing here they cannot get from a tutorial.

Three things to set up before you start

1. Two windows side by side: VS Code with the starter, and a browser on starter/index.html. Not a projector showing

one at a time.

2. The browser console open (F12) and left open all session. You will use it in steps 4, 5, 6, 10 and 11.

3. The finished app in a second browser tab, so you can flip to it when somebody asks what it is supposed to look like.

0 Open the finished app and the three empty files

BOTH · 3 minutes

SAY

- Start from the end. Open reference/index.html, make one payment, and let them see what they are aiming at.
- Then tick the layer toggle in the header. Blue is CSS, purple is JavaScript. Say the sentence you will repeat all session: CSS decides what it looks like, JavaScript decides what is true.
- Now close it and open the starter. index.html is finished - nobody writes markup today. style.css has nine TODOs, app.js has twelve.

SHOW Finished app, then the layer view, then the starter's empty files side by side in VS Code.

CHECK Everyone can open index.html in a browser and see the unstyled form. If the page is blank, they opened the wrong file.

1 Tokens first, then the page basics

CSS · C1 + C2 · 4 minutes

SAY

- Name every colour once, at the top, and never type a hex code again below this block.
- Ask them why --brand instead of #c8102e in forty places. The answer is the one-line change when Marketing picks a new red.
- box-sizing: border-box on everything is the line that stops padding breaking your widths. Set it once and forget it.

TYPE `style.css`

```
:root {
  --bg:        #f6f7f9;
  --surface:   #ffffff;
  --ink:       #15181d;
  --muted:     #6b7280;
  --line:      #e3e6ea;
  --brand:     #c8102e;
  --brand-ink: #ffffff;
  --ok:        #1f7a45;
  --ok-bg:     #e9f6ee;
  --warn:      #9a6100;
  --warn-bg:   #fdf3e2;
  --bad:       #b3261e;
  --bad-bg:    #fdecec;
  --radius:    14px;
  --shadow:    0 1px 2px rgba(16,24,40,.06), 0 8px 24px rgba(16,24,40,.06);
  --font:      system-ui, -apple-system, "Segoe UI", Roboto, Arial, sans-serif;
  --mono:      ui-monospace, "Cascadia Mono", Consolas, monospace;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  background: var(--bg);
  color: var(--ink);
  font-family: var(--font);
  font-size: 16px;
  line-height: 1.5;
}

body {
  margin: 0;
  background: var(--bg);
  color: var(--ink);
  font-family: var(--font);
  font-size: 16px;
  line-height: 1.5;
}
```

SHOW Reload. The page is still ugly, but the font and the background changed - proof the stylesheet is actually linked.

CHECK If nothing changed, the <link> tag path is wrong. Open the Network tab and look for the 404 on style.css.

2 Layout: a container, a grid, and one card rule

CSS · C3 + C4 + C5 · 4 minutes

SAY

- Three rules turn a stack of text into an application.
- .wrap centres the content and keeps it off the edges on a phone.
- .grid is the two-column split. Point out that 1.05fr and 0.95fr is just 'the left one slightly wider' - fr units are proportions, not pixels.
- .card is written once and used by every panel on the page. That is the whole idea of a class.

TYPE `style.css`

```
.wrap { max-width: 1120px; margin: 0 auto; padding: 0 20px; }

/* ---- top bar ---- */
.topbar {
  background: var(--surface);
  border-bottom: 1px solid var(--line);
  position: sticky; top: 0; z-index: 10;
}

.grid {
  display: grid;
  grid-template-columns: 1.05fr 0.95fr;
  gap: 20px;
  padding-top: 24px;
  padding-bottom: 32px;
  align-items: start;
}

.card {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 22px;
  position: relative;
}
```

SHOW Reload. The form and the receipt sit side by side in two white cards. This is the moment it starts looking like software.

CHECK Narrow the window hard. It will look wrong - that is C9's job later, so leave it.

3 Make the form controls look like this decade

CSS · C6 · 4 minutes

SAY

- font: inherit is the single most valuable line here. Without it, inputs keep the browser's default 13px Arial and look pasted on.
- Spend a minute on :focus-visible. Tab through the form with and without it. Somebody navigating by keyboard cannot use the form without that ring, and it is two lines.

TYPE `style.css`

```
input[type="text"], select {
  width: 100%;
  padding: 11px 13px;
  font: inherit;
  font-size: 15px;
  color: var(--ink);
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 10px;
  transition: border-color .15s, box-shadow .15s;
}
input:focus-visible, select:focus-visible, button:focus-visible {
  outline: none;
  border-color: var(--brand);
  box-shadow: 0 0 0 3px color-mix(in srgb, var(--brand) 22%, transparent);
}
```

SHOW Tab through the form and watch the focus ring move.

CHECK Ask the room to put their mouse down and reach the submit button with the keyboard only.

4 The first function, and the trap inside it

JAVASCRIPT · J1 · 5 minutes

SAY

- Open the console and run Number('') before showing the function. It is 0. Let that sit for a second.
- So an empty amount box becomes zero, and zero passes any test that only asks 'is this a number'. That is why the blank check comes first and the conversion second.
- This function takes text and returns a number. It touches nothing else, which is why we can test it in the console right now.

TYPE app.js

```
function toAmount(text) {
  if (typeof text !== 'string' || text.trim() === '') { return NaN; }
  return Number(text);
}
```

SHOW In the console: toAmount('48.20'), toAmount(''), toAmount('twelve'). Three answers, three different reasons.

CHECK Number.isNaN(toAmount('')) is true. If it is false, the blank check is missing or in the wrong order.

5 Validation: is this data usable at all?

JAVASCRIPT · J2 · 7 minutes

SAY

- Validation never asks what the company allows. It only asks whether the request makes sense as data. Keep saying that - it is the idea the whole task is built on.
- Notice it returns an object, not true or false. That is how the page knows WHICH field to turn red and WHAT to say under it.
- Each check returns the moment it finds a problem. Later checks cannot run on data you already know is broken.

TYPE app.js

```
function validate(form) {
  const errors = {};

  if (!findAccount(form.account)) {
    errors.account = 'Choose an account to pay from.';
  }

  const payee = form.payee.trim();
  if (payee === '') {
    errors.payee = 'Payee is required.';
  } else if (payee.length < 2) {
    errors.payee = 'Payee must be at least 2 characters.';
  } else if (payee.length > POLICY.maxPayeeLength) {
    errors.payee = 'Payee must be ' + POLICY.maxPayeeLength +
                   ' characters or fewer.';
  }

  const amount = toAmount(form.amount);
  if (form.amount.trim() === '') {
    errors.amount = 'Enter an amount.';
  } else if (Number.isNaN(amount)) {
    errors.amount = 'Amount must be a number.';
  } else if (amount <= 0) {
    errors.amount = 'Amount must be greater than 0.';
  } else if (Math.round(amount * 100) !== amount * 100) {
    errors.amount = 'Amount can have at most 2 decimals.';
  } else if (amount > POLICY.maxPerPayment) {
    errors.amount = 'Single payments are capped at ' +
                    money(POLICY.maxPerPayment) + '.';
  }

  if (!CATEGORIES.includes(form.category)) {
    errors.category = 'Choose a category.';
  }

  if (form.reference.length > POLICY.maxReferenceLength) {
    errors.reference = 'Reference must be ' + POLICY.maxReferenceLength +
                       ' characters or fewer.';
  }

  return errors;
}
```

SHOW In the console: validate({account:'CHK-4471', payee:'', amount:'0', category:'', reference:''}). Three keys come back.

CHECK A good request returns {} - an empty object, not undefined.

ASK THE ROOM *Why does this function not know about the account balance?*

6 Read the form, and show the errors

JAVASCRIPT · J7 + J8 · 6 minutes

SAY

- Here is the line between the layers, in two functions.
- readForm goes to the page and collects five values. showErrors goes back to the page and sets text and class names.
- showErrors does not choose a colour. It adds the class 'invalid' and CSS decides what that means. If you ever type a colour in app.js, stop.

TYPE app.js

```
function readForm() {
  return {
    account:   $('#account').value,
    payee:     $('#payee').value,
    amount:    $('#amount').value,
    category:  $('#category').value,
    reference: $('#reference').value
  };
}

function showErrors(errors, touchedOnly) {
  for (const name of FIELDS) {
    const input = $('#' + name);
    const slot = document.querySelector('.err[data-for="' + name + '"]');
    const message = errors[name];
    const show = message && (!touchedOnly || input.dataset.touched === 'yes');

    slot.textContent = show ? message : '';
    input.classList.toggle('invalid', Boolean(show));
    input.classList.toggle('valid',
      !message && input.value.trim() !== '' && name !== 'reference');
  }
}
```

SHOW Nothing visible yet - the classes are being set, but CSS does not know what they look like. That is the next step, and the order is deliberate.

CHECK In the console: showErrors({payee:'test message'}) and then inspect the payee input. It has class 'invalid'.

7 Now tell CSS what 'wrong' looks like

CSS · C7 · 3 minutes

SAY

- Three rules, and the invisible work from the last step becomes visible.
- .err:empty { display: none } is the neat part - the message element is always in the HTML, and it only takes up space when JavaScript has put text in it. No layout jump.

TYPE `style.css`

```
input.invalid, select.invalid { border-color: var(--bad); background: var(--bad-bg); }
input.valid, select.valid     { border-color: var(--ok); }

.err { color: var(--bad); font-size: 13px; margin: 6px 0 0; min-height: 1px; }
.err:empty { display: none; }
```

SHOW Reload, run showErrors in the console again. Red border, red message. Same JavaScript as a minute ago - only the stylesheet changed.

CHECK Clear the error and the space collapses. If it does not, the :empty rule is missing.

8 Two questions the form cannot answer

JAVASCRIPT · J3 + J4 · 6 minutes

SAY

- These are the first functions that need HISTORY, not just the form.
- postedToday loops every transaction and adds up today's. This is the accumulator pattern from Week 7 - start outside the loop, add inside.
- recentDuplicate is Week 13's duplicate-submission problem arriving early. Same payee, same amount, inside a minute: almost certainly one payment sent twice.
- Say plainly: no amount of checking the form could answer either of these. That is why the data lives in an array.

TYPE app.js

```
function postedToday(code) {
  const midnight = new Date();
  midnight.setHours(0, 0, 0, 0);
  let total = 0;
  for (const t of transactions) {
    if (t.account === code && t.status === 'POSTED' &&
        t.created >= midnight.getTime()) {
      total = total + t.amount;
    }
  }
  return total;
}

function recentDuplicate(code, payee, amount) {
  const cutoff = Date.now() - POLICY.duplicateWindowMs;
  for (const t of transactions) {
    if (t.account === code && t.payee === payee &&
        t.amount === amount && t.created >= cutoff) {
      return t;
    }
  }
  return null;
}
```

SHOW Nothing on screen. Both are console-testable once there are a few transactions.

CHECK postedToday('CHK-4471') is 0 before any payments.

9 The business rules - and the order argument

JAVASCRIPT · J5 · 8 minutes

SAY

- The data is already known to be good. These decide what happens to it.
- Read the four branches out loud in order. Then ask the room: a $9,500 payment from an account holding $4,100, sent twice in ten seconds - which message should the user see?
- There is no single correct answer, but there is a correct method: pick the one most useful to the person reading it. Put a comment above the chain saying why you chose that order.
- This is the highest-value conversation in the whole demo. Do not rush it.

TYPE `app.js`

```
function applyRules(form) {
  const account = findAccount(form.account);
  const amount = toAmount(form.amount);
  const payee = form.payee.trim();

  if (amount > account.balance) {
    return { status: 'DECLINED', reason: 'INSUFFICIENT_FUNDS',
             message: 'Balance is ' + money(account.balance) +
                      '. This payment is ' + money(amount) + '.' };
  }

  const spent = postedToday(account.code);
  if (spent + amount > POLICY.maxPerDay) {
    return { status: 'DECLINED', reason: 'DAILY_LIMIT',
             message: 'The daily limit is ' + money(POLICY.maxPerDay) +
                      '. ' + money(POLICY.maxPerDay - spent) + ' left today.' };
  }

  if (recentDuplicate(account.code, payee, amount)) {
    return { status: 'HELD', reason: 'POSSIBLE_DUPLICATE',
             message: 'The same payee and amount went through less than a ' +
                      'minute ago. Held for review.' };
  }

  if (amount >= POLICY.reviewAtOrAbove) {
    return { status: 'HELD', reason: 'REVIEW_REQUIRED',
             message: 'Payments of ' + money(POLICY.reviewAtOrAbove) +
                      ' or more are reviewed before release.' };
  }

  return { status: 'POSTED', reason: 'OK',
           message: 'Payment posted to the account.' };
}
```

SHOW Swap the first two branches live, reload, and send $9,500 from the aid account. The answer changes. Nothing was broken - the order IS the policy.

CHECK Put it back. Every branch returns the same shape: status, reason, message.

ASK THE ROOM *Which of these four rules could you have written in CSS? None. Why not?*

10 One payment, end to end

JAVASCRIPT · J6 · 6 minutes

SAY

- Validate, then apply the rules, then write it down. A rejected request never reaches the rules - look at the early return.
- Three statuses, not two. REJECTED means the data was unusable. HELD means the data was fine and policy stopped it. They are different things and the user deserves to know which happened.
- unshift puts the newest transaction at the front of the array, which is why the ledger needs no sorting later.

TYPE app.js

```
function processPayment(form) {
  const errors = validate(form);
  if (Object.keys(errors).length > 0) {
    return { ok: false, errors: errors };
  }

  const decision = applyRules(form);
  const account = findAccount(form.account);
  const amount = toAmount(form.amount);

  counter = counter + 1;
  const txn = {
    id: 'TXN-' + new Date().getFullYear() + '-' +
        String(counter).padStart(4, '0'),
    created: Date.now(),
    account: account.code,
    payee: form.payee.trim(),
    amount: amount,
    category: form.category,
    reference: form.reference.trim(),
    status: decision.status,
    reason: decision.reason,
    message: decision.message
  };

  transactions.unshift(txn);
  if (txn.status === 'POSTED') {
    account.balance = Math.round((account.balance - amount) * 100) / 100;
  }

  return { ok: true, transaction: txn };
}
```

SHOW In the console: processPayment({account:'CHK-4471', payee:'Bursar', amount:'840.25', category:'tuition', reference:''}). A whole transaction object comes back.

CHECK Run it again with amount '9999999' and watch it come back ok:false with an errors object instead.

11 Put the answer on the page

JAVASCRIPT · J9 + J10 + J11 · 7 minutes

SAY

- Three render functions, and none of them decide anything. They take data that already exists and turn it into HTML.
- renderAccounts is called again after every payment, which is why the balance on screen is never stale.
- In renderLedger, point at the totals loop. That is the same accumulator from step 8, except now the room can see the number change.

TYPE app.js

```
function renderReceipt(txn) {
  const cls = txn.status.toLowerCase();
  $('#result').innerHTML =
    '<div class="receipt">' +
      '<h2>Receipt</h2>' +
      '<div class="verdict"><span class="pill ' + cls + '">' +
        txn.status + '</span></div>' +
      '<p>' + txn.message + '</p>' +
      '<div class="rows">' +
        row('Reference', '<span class="mono">' + txn.id + '</span>') +
        row('Paid to', txn.payee) +
        row('Amount', '<span class="mono">' + money(txn.amount) + '</span>') +
        row('From', txn.account) +
        row('Category', txn.category) +
        (txn.reference ? row('Note', txn.reference) : '') +
        row('Time', new Date(txn.created).toLocaleTimeString()) +
      '</div>' +
    '</div>';
}

function renderLedger() {
  const body = $('#ledger');

  if (transactions.length === 0) {
    body.innerHTML = '<tr><td colspan="4" class="muted">' +
                     'No payments yet.</td></tr>';
  } else {
    let html = '';
    for (const t of transactions.slice(0, 8)) {
      html += '<tr>' +
        '<td class="mono">' + t.id + '</td>' +
        '<td>' + t.payee + '</td>' +
        '<td class="r mono">' + money(t.amount) + '</td>' +
        '<td><span class="pill ' + t.status.toLowerCase() + '">' +
            t.status + '</span></td></tr>';
    }
    body.innerHTML = html;
  }

  /* the accumulator pattern from Week 7, on screen */
  let posted = 0, held = 0, total = 0;
  for (const t of transactions) {
    if (t.status === 'POSTED') { posted++; total = total + t.amount; }
    else if (t.status === 'HELD') { held++; }
  }
  $('#totals').innerHTML =
    tile(transactions.length, 'submitted') +
    tile(posted, 'posted') +
    tile(money(total), 'paid out');
  void held;
}
```

SHOW Still nothing until the submit handler exists - these are being defined, not called.

CHECK Call renderLedger() by hand in the console. The empty-state row appears.

12 One pill, three meanings

CSS · C8 · 3 minutes

SAY

- One shape rule, three colour variants. JavaScript writes the class name from the status; CSS decides what each one looks like.
- This is the clearest example of the split on the whole page. The word DECLINED is JavaScript's. The fact that it is red is CSS's.

TYPE `style.css`

```
.pill { display: inline-block; padding: 3px 10px; border-radius: 999px;
        font-size: 12px; font-weight: 700; letter-spacing: .02em; }
.pill.posted   { background: var(--ok-bg);   color: var(--ok); }
.pill.held     { background: var(--warn-bg); color: var(--warn); }
.pill.declined { background: var(--bad-bg);  color: var(--bad); }
```

SHOW Nothing yet - no receipt has been drawn. Next step lights it up.

CHECK Three variants, all readable. Make them check the contrast, not just the vibe.

13 Wire it up - and watch the whole thing come alive

JAVASCRIPT · J12 · 7 minutes

SAY

- preventDefault() is the line that stops the page reloading. Without it the browser does what it did in 1998 and everything you just wrote is thrown away.
- The blur handler is why the form does not shout at you while you are still typing. It only complains about a field once you have left it.
- The setTimeout is honest theatre - it gives the busy state something to be busy during. In Week 10 that pause becomes a real request to a real server, and nothing else in this file changes.

TYPE app.js

```
document.addEventListener('DOMContentLoaded', function () {
  // ... the blur and input handlers ...

  $('#payForm').addEventListener('submit', function (event) {
    event.preventDefault();          // no reload - we handle it here

    for (const name of FIELDS) { $('#' + name).dataset.touched = 'yes'; }

    const form = readForm();
    const errors = validate(form);
    showErrors(errors, false);

    if (Object.keys(errors).length > 0) {
      $('#result').innerHTML =
        '<h2>Receipt</h2><p class="muted">Nothing was sent. ' +
        Object.keys(errors).length +
        ' field(s) need fixing first.</p>';
      $('#' + Object.keys(errors)[0]).focus();
      return;
    }

    /* A short pause so the pending state is visible. A real app would
       be waiting on a server here - that is Week 10. */
    const button = $('#submitBtn');
    button.classList.add('busy');
    button.disabled = true;

    window.setTimeout(function () {
      const result = processPayment(form);
      button.classList.remove('busy');
      button.disabled = false;

      renderReceipt(result.transaction);
      renderAccounts();
      renderLedger();

      if (result.transaction.status === 'POSTED') {
        $('#payForm').reset();
        $('#refCount').textContent = '0';
        for (const name of FIELDS) {
          $('#' + name).dataset.touched = '';
          $('#' + name).classList.remove('valid', 'invalid');
        }
        renderAccounts();
      }
    }, 450);
  });
```

SHOW Fill in the form and press Send payment. Receipt, balance drop, ledger row, totals. The app is finished.

CHECK Try all three outcomes: 840.25 posts, 6200 is held, 9500 from the aid account is declined.

14 Make it survive a phone

CSS · C9 · 3 minutes

SAY

- Three lines. The HTML does not change and the JavaScript does not change - only the stylesheet has an opinion about screen width.
- Resize the window slowly and let them watch it snap to one column.

TYPE `style.css`

```
@media (max-width: 900px) {
  .grid { grid-template-columns: 1fr; }
  .two  { grid-template-columns: 1fr; }
  .totals { grid-template-columns: 1fr 1fr; }
}
```

SHOW Drag the window narrow. Two columns become one.

CHECK Open DevTools device mode and try an iPhone SE. Nothing should overflow sideways.

15 The two stretch pieces worth demoing

BOTH · S1 + S3 · 5 minutes

SAY

- Dark mode is eight lines because step 1 was done properly. Redefine the tokens, change nothing else. If a student hard-coded colours below :root, this is where it hurts - and that is the lesson.
- The layer view is one CSS block plus one line of JavaScript. Show the toggle, then show how little code it took.

TYPE `style.css`

```
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0f1216; --surface: #171b21; --ink: #e9edf2; --muted: #9aa4b2;
    --line: #262c35; --ok-bg: #11291c; --warn-bg: #2a2110; --bad-bg: #2b1414;
    --ok: #5fd08a; --warn: #e3b25f; --bad: #ff8b84;
    --shadow: 0 1px 2px rgba(0,0,0,.4), 0 10px 30px rgba(0,0,0,.35);
  }
}

/* and the layer view, in full: */
[data-layer]::before {
  content: attr(data-layer-note);
  position: absolute; top: -11px; left: 14px;
  font-size: 11px; font-weight: 700; letter-spacing: .03em;
  padding: 2px 9px; border-radius: 999px;
  opacity: 0; pointer-events: none; transition: opacity .2s;
}
[data-layer="css"]::before { background: #2563eb; color: #fff; }
[data-layer="js"]::before  { background: #a855f7; color: #fff; }

body.show-layers [data-layer] { position: relative; }
body.show-layers [data-layer]::before { opacity: 1; }
body.show-layers [data-layer="css"] { outline: 2px dashed #2563eb; outline-offset: 3px; }
body.show-layers [data-layer="js"]  { outline: 2px dashed #a855f7; outline-offset: 3px; }
```

SHOW Switch the OS to dark mode, then tick the layer toggle.

CHECK If dark mode looks broken, somebody wrote a colour outside the tokens. Find it together - it takes thirty seconds and the point lands permanently.

Appendix - the numbers to demo with

Keep these on a sticky note. They hit every branch of the rules in order.

Amount	From	Outcome	What it proves
840.25	Checking	POSTED	The happy path, and the balance drops
0 / -40 / twelve	any	Rejected at validation	Three different messages, no rules ran
6200	Checking	HELD	Valid data, policy stopped it
9500	Financial Aid	DECLINED	Over the balance
840.25 twice, fast	Checking	HELD	Duplicate inside 60 seconds
10500	any	Rejected at validation	Over the per-payment cap

Appendix - questions they will ask

Why not just use the browser's built-in validation (required, min, type=number)?

Use it as well - it is free. But it cannot express 'over the daily limit' or 'duplicate of something sent a minute ago', and its messages cannot be styled to match the app. We turned it off with novalidate so the lesson is visible.

Could the rules go in the CSS?

No, and it is worth answering seriously. CSS can react to states a browser already knows (:invalid, :empty), but it cannot add numbers, look at history, or decide anything. The moment you need an if, you are in JavaScript.

Why does the data vanish when I reload?

Because it only lives in a JavaScript array. That is honest for this week. Persistence is Week 12, and the fix is a server, not a cleverer array.

Is this secure? Can't I just edit the JavaScript and pay myself?

Yes, and you should say so plainly. Everything here is for usability. A real system re-checks every rule on the server, because anything in the browser belongs to the user. That is Week 11.

Can I use a framework / library for this?

Not for this task. The point is seeing the DOM calls and the decisions with nothing in between. You will meet the abstraction later and it will make more sense for having done it by hand once.

The sentence to repeat until they say it back

CSS decides what it looks like. JavaScript decides what is true.

Everything in this demo is a worked example of that one line - and it is also the thing the code defence will test, because a student who cannot say which file a decision belongs in did not write the file.